TOPIC: Video Chatting Website

https://vid-at.herokuapp.com/

Abstract

Collaboration has caught tremendous importance with the rapid rise of the pandemic situation of 2020. Many companies are switching to Remote work during this quarantine. People are staying at home and can't see their friends and communicate with their colleagues as they used to. This has led to a fast increase in the popularity of video chat apps.

Today, everyone uses some sort of video calling app to socialize. Native applications are more in demand rather than Video chat online applications. Although there are some existing Progressive Web Apps (PWA's) webapps and websites, they have not gained much popularity because of some limitations for commercial purposes. What if we could build a website for video chatting between peers? What if we could send data streams between two browsers?

We can establish a session between two peer client browsers and send data over HTTP protocol. The application being hosted over the internet, we save our internal device memory space. And with the browsers being available by default on all desktop and mobile devices we can access it anytime and anywhere.

Video chat website can increase employee productivity. Employees who usually communicate via email show a slow pace of work. With our video chat webapp, issues can be solved faster and in real time.

Table of Contents

Sr. no.	Topic	Pg. no.
1.	Introduction	4
2.	Native Device Apps	5
3.	Website using WebRTC	6
4.	Architecture	7
5.	Methodology	8
6.	Implementation	9
7.	Conclusion	10

Introduction

We will integrate WebRTC Google's open-source project developed using JavaScript and C++. It is used for peer to peer connection on the web, meaning that your browser, for example, is able to connect to another browser and share different kinds of data between them, like video, audio stream, or just some chunk of JSON data.

To support communication between two peer browsers, we will create an active session. Any other client accessing the application will be redirected to "Server Busy" error page and requested to visit the site later. There will be a toggle mute facility provided to the user to shut down his peer's audio. When one of the client exits the browser the active session will be destroyed and the connection will be available for other client to connect.

The backend logic will be done in JavaScript with its runtime Node.js having powerful cross-platform and robust environment for development. To make our development faster and flexible, Express JS a Node.js framework will be used. Our application is also accompanied with Socket.io a full-stack framework of JavaScript library that focuses on creating real-time applications.

Lastly, the deployment of our website will be done on cloud with a platform as a service (PaaS) – Heroku. The application will go live and free to access for everyone online.

Native Device Apps

A native application is a software program that is developed for use on a particular platform or device.

Because a native app is built for use on a particular device and its OS, it has the ability to use device-specific hardware and software. Native apps can provide optimized performance and take advantage of the latest technology, such as a Camera or GPS, compared to web apps or mobile cloud apps developed to be generic across multiple systems.

Although having applications installed on your device provided more features to access the device resources. There is a limitation of cross-browser compatibility for online platforms. Applications like Zoom, Google Meet, Cisco Webex Meeting, etc. have more popular native app versions than the traditional websites. The most prominent common reason is ease of access and less data consumption.

Why should we use native application for Video Chatting?

- Client side load and processing is heavy
- Cross platform support (IOS, Android, Linux, etc.)
- Advanced functions – interact with other apps, push notifications, etc.
- Higher computing performance
- More security options
- Reliability and robustness
- Device portability

Website using WebRTC

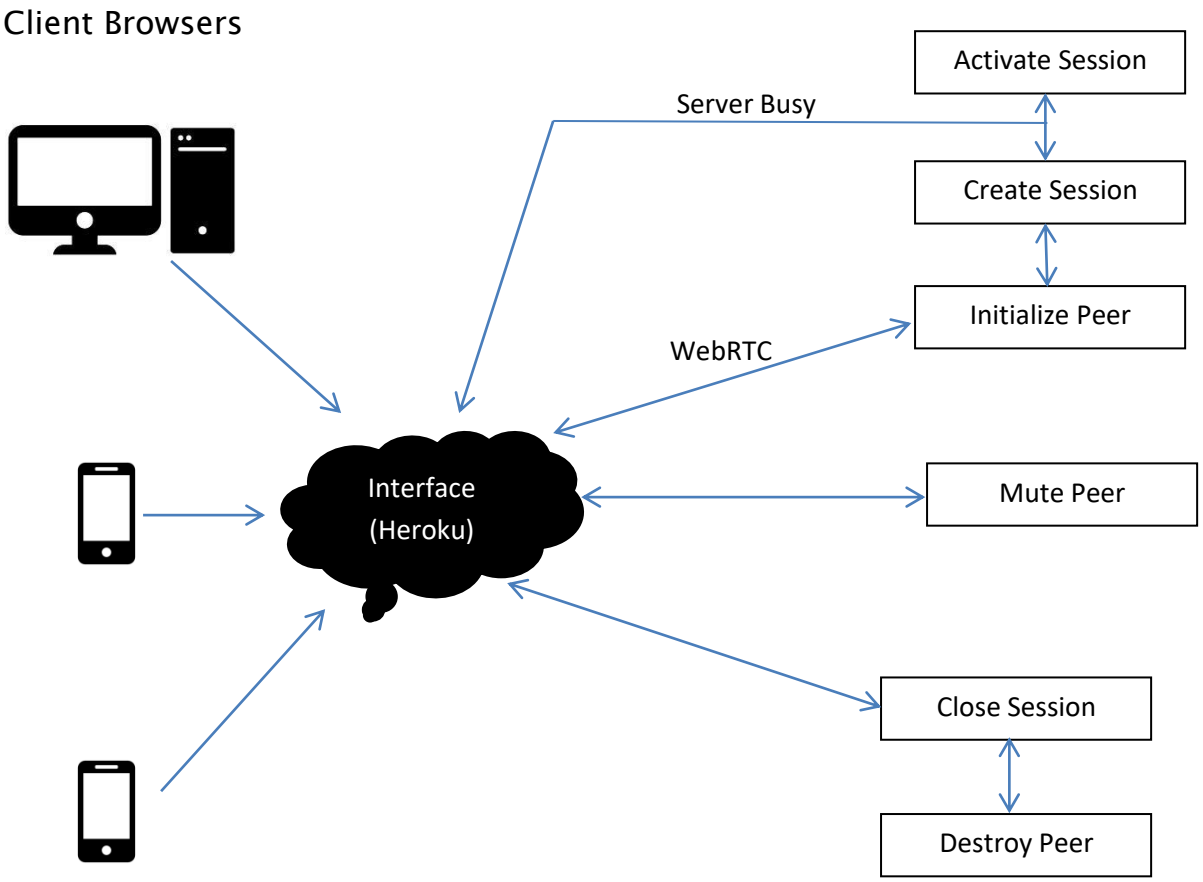
Web Real Time Communication (WebRTC) provides an equivalent performance just as a native application. Video chat webapps are efficient, easy to handle and provide an optimal way to free up space. With the emergence of **Progressive Web Apps** even their limitations are surpassed. PWA's also called as Hybrid apps can now even access device file resources, send notifications and even create a home screen shortcut for faster access.

Using cookies and browser cache we can store user's data and ask permissions. For example: Storing camera and mic access permissions in browser buffer to provide app-like functionality. Many have questioned if PWA will replace native apps in the future. In addition, users are more likely to trust an app than a URL, as it needed to pass the App Store's security requirements before getting published.

Why should we use web application for Video Chatting?

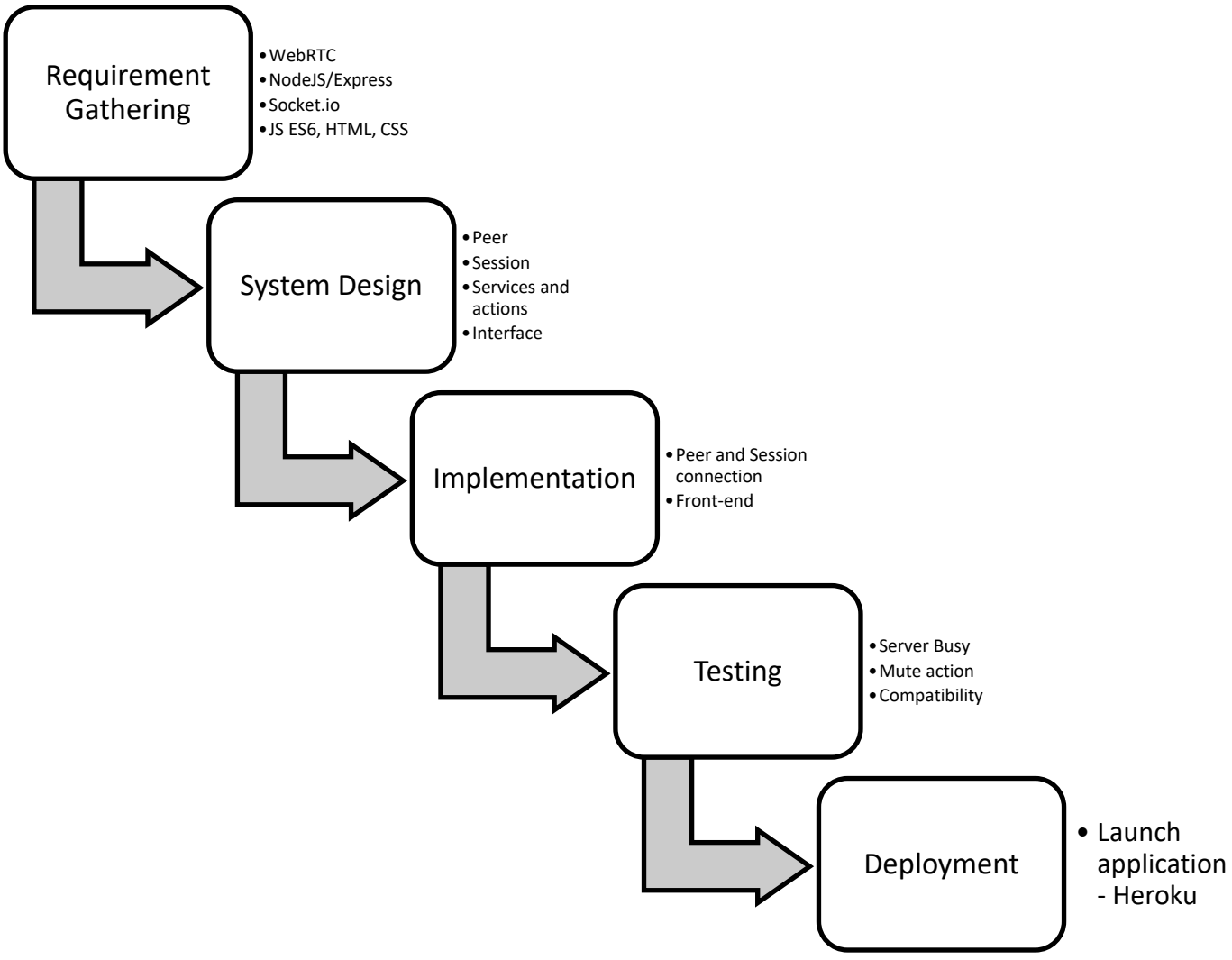
- Use web-technologies – HTML, CSS, JavaScript
- Cheaper than native app
- Faster to build and update
- No need of building different versions for different platforms
- Cross-Browser compatibility
- Search Engine Optimization for Brand
- Security over HTTPS avoiding tampering
- Easy Installation without download
- Less client-side processing needed
- Decreased memory consumption

Architecture



Methodology

We have implemented the Waterfall methodology to implement the Video-Chat application website and deploy it on Heroku. This approach was considered because of the requirements availability in advance.



Implementation

Backend Technologies used:

- WebRTC
- Node.js v12.16.1 & npm 6.13.4
- Express JS
- Socket.io

Front-end Technologies used:

- JavaScript ES6
- HTML5
- CSS

Entities:

- Peer – Another client browser instance that we want to establish a connection with for data streaming.
- User – Refers to Client itself accessing the website.
- Session – Create a common medium for clients to communicate.

Methods:

- CreatePeer() – Initialize a client's request as Peer and create a Video object ready for streaming on the website.
- SendOffer() – The former initialized peer sends offer to another peer to connect to the active session
- SendAnswer() – The later initialized peer sends answer of acknowledgement of offer back to the former initialized peer.
- SessionActive() – Establishes a secure connection over HTTPS and prevents third peer client from disturbing the active session.
- Disconnect() – Destroys the peer object on browser close event and remove the video component from page.

Conclusion

With the help of Google's WebRTC library we can stream the video data stream over the internet easily using HTTP's support for security. We created a session for two client browsers sending an offer and answer. We also handled any additional peer request for connection. In future, we can convert the website to function as an application using PWA. Additionally, we can increase the project's scope with more than two clients and support for multiple sessions handling with load balancing at the same time.

Research shows that 62% of executives agree that relative to audio-conferencing, video conferencing significantly improves the quality of communication, a figure that rises to 73% among high-growth companies. In addition, 50% of those surveyed believe video conferencing also improves the degree of understanding.